ProjectFlow

Project Management System (Web + Mobile)

The Complete Guide — explained in plain English, with the code

Written for someone with no technical background who must still answer questions about the code in a review session.

Live website	https://projectflow-web-tau.vercel.app
Live API (backend)	https://projectflow-api-virid.vercel.app/api
Source code (public)	https://github.com/AdityaParthasarathy/project-management-system
Demo login	demo@projectflow.test / DemoPass#2026
Local website	http://localhost:3000 (API on http://localhost:5000/api)

How to use this guide

You will have three windows open: the **running website**, the **code**, and **this guide**. The guide is organised so you can look at a screen in the website, find the matching part here, and then find the matching file in the code. Every section tells you **which file** to open.

1. The project in one minute (and a story that makes it easy to remember)
2. The big picture: the four pieces and how they talk
3. Jargon decoder (every technical word explained simply)
4. Folder-by-folder map of the code
5. The database (the "filing cabinet")
6. The backend, file by file (the "kitchen")
7. Follow one click all the way through the system
8. The website (the "restaurant front")
9. The mobile app
10. Security: what protects the system and where
11. Testing: how we know it works
12. Deployment: how it got onto the internet
13. Requirement checklist: the assignment PDF vs. what we built
14. A 5-minute demo script, with what to say
15. 50 questions the reviewer might ask, with simple answers
16. Honest limitations and what you can say about them
17. Cheat sheet (URLs, commands, logins)

> **The golden rule for the review:** you do not need to memorise code. You need to be able to say *what each part is for, why it exists, and where it lives*. Section 15 gives you ready-made answers. If you get a question you can't answer, it is perfectly fine to say: "That's handled in this file — let me open it," and read the comments and names; the code is written to be readable.

1. The project in one minute

What it is: an application where people sign up, create **projects** (like "Website Redesign"), and put **tasks** inside those projects (like "Design homepage"). Each task has a priority (Low / Medium / High) and a status (Pending / In Progress / Completed). A **dashboard** shows live totals. You can use it on a **website** or on an **Android phone app**, and both show exactly the same data because both talk to the **same backend and the same database**.

The restaurant story (use this to explain it to anyone)

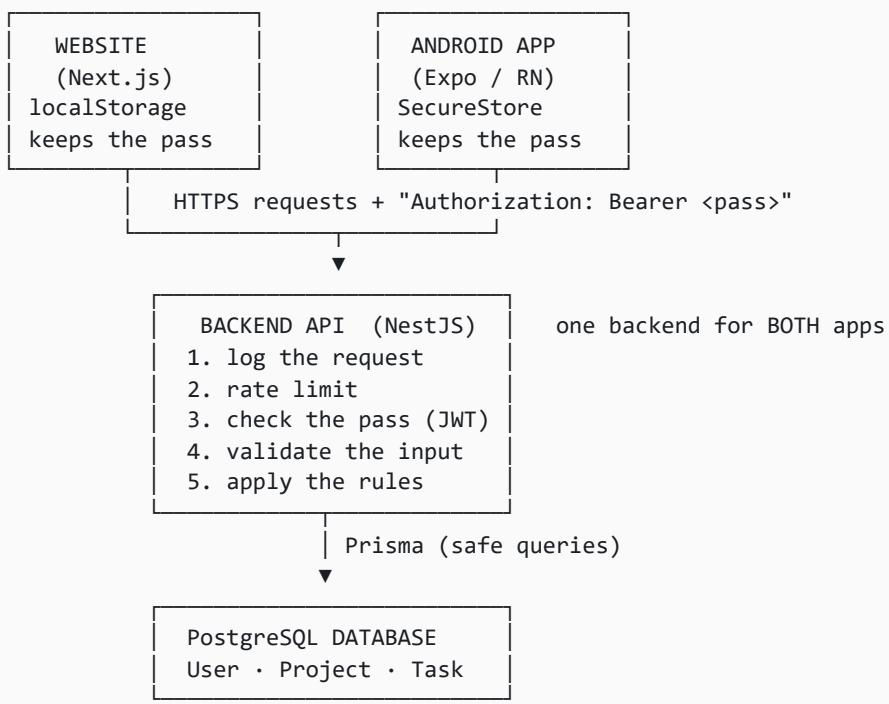
In the restaurant	In our system	Where in the code
The dining room — customers see the menu and order	**Website** (frontend) and **Android app** (mobile). They only show things and collect clicks.	`frontend/`, `mobile/`
The waiter — carries orders to the kitchen and food back	**The API**: a list of agreed "requests" (like *GET /api/tasks*) that carry data between screens and the backend, in a format called JSON.	URLs in `backend/src/*/*.controller.ts`
The kitchen — where the real work and the rules are	**Backend** (NestJS). Checks who you are, checks your input, applies rules, talks to the database.	`backend/src/`
The pantry / filing cabinet — where everything is stored	**Database** (PostgreSQL) with three drawers: Users, Projects, Tasks.	`backend/prisma/schema.prisma`
The ID card at the door	**JWT token**: after login you receive a signed digital pass; you show it with every request.	`backend/src/auth/`
The bouncer who limits how often one person can knock	**Rate limiter**: stops password-guessing by blocking too many attempts.	`backend/src/common/rate-limit.ts`

Technology chosen (and the one-line reason for each)

Part	Technology	Why / what to say
Website	**Next.js** (built on React)	Assignment allows "React or Next.js". Next.js is React plus easy building and hosting.
Mobile app	**Expo / React Native** (TypeScript)	Allowed option. One codebase builds a real Android app; Expo makes building the APK easy.
Backend	**NestJS** (Node.js)	Allowed option. It forces a tidy structure: modules, controllers, services.
Database	**PostgreSQL**	Allowed option. A relational database: tables linked by keys.
Database tool	**Prisma** (an "ORM")	Lets the code talk to the database safely without hand-writing SQL; this is what protects against SQL injection.

Passwords	**bcrypt**	Industry-standard way to scramble passwords so they can't be read back.
Login passes	**JWT** (JSON Web Token)	Standard "digital pass" that works for both web and mobile.
Language	**TypeScript**	JavaScript with types; catches mistakes before running.

2. The big picture



Key sentence for the review: "The web app and the mobile app are just two different screens on the same backend. Neither one talks to the database directly. That is why a change made on the phone shows up on the website after a refresh."

Where each piece is running right now

Piece	Hosted (public internet)	On your computer (local)
Website	https://projectflow-web-tau.vercel.app (Vercel)	http://localhost:3000
Backend API	https://projectflow-api-virid.vercel.app/api (Vercel serverless)	http://localhost:5000/api
Database	Hosted PostgreSQL (Prisma Postgres)	PostgreSQL on your PC, database `project_management`

> **Important:** the local website (localhost:3000) uses the *local* backend and *local* database; the Vercel website uses the *hosted* backend and database. They are two separate sets of data. The demo user `demo@projectflow.test` exists in both.

3. Jargon decoder

Word	Plain-English meaning
Frontend	What the user sees and clicks (the website).
Backend	The hidden program on a server that holds the rules and talks to the database.
API	A menu of requests the backend understands. Each has an address (URL) and a verb.
REST	A common style of API: GET = read, POST = create, PUT = update, DELETE = delete, on addresses like `/projects/5`.
JSON	The text format data travels in, e.g. `{"name":"Task 1","priority":"HIGH"}`.
Endpoint	One item on the API menu, e.g. `POST /api/tasks`.
Database / table / row	Organised storage. A table is a spreadsheet (e.g. Task); each row is one record (one task).
Primary key (PK)	A unique number for each row (`id`).
Foreign key (FK)	A column that points to a row in another table (a Task's `projectId` points to its Project). The database refuses bad links.
Cascade delete	Deleting a project automatically deletes its tasks; deleting a user deletes their projects and tasks.
Enum	A column that can only hold one of a fixed list, e.g. priority ∈ LOW, MEDIUM, HIGH.
ORM / Prisma	A translator between code and database. You write `prisma.task.findMany(...)`; it produces safe SQL.
Migration	A saved script that builds or changes the database tables, so anyone can recreate the same database with one command.
SQL injection	An attack where a user types database commands into a form. Prevented because Prisma sends user input as *values*, never as part of the command text.
Hashing / bcrypt	One-way scrambling. We store the scrambled password; at login we scramble what you typed and compare. Nobody (not even us) can unscramble it.
JWT	A signed pass containing your user id and an expiry time. The server signs it with a secret key; any tampering breaks the signature. After one day it stops working.
Bearer token	How the pass is sent: header `Authorization: Bearer <token>`.
Authentication	"Who are you?" (login).
Authorization	"What are you allowed to touch?" (you only see your own projects/tasks).
Guard	NestJS term for a checkpoint placed in front of a route (e.g. "must have a valid token").
Middleware	Code that runs on every request before the route (our request logger).
DTO	"Data Transfer Object": a class that describes what a request body must look like (required fields, formats). The validator rejects anything else.

Validation	Checking input is acceptable (email looks like an email, enum value is valid, name not empty).
CORS	Browser safety rule: a website may call an API only if the API lists that website as allowed. We allow our Vercel site and localhost.
Rate limiting	Capping how many requests one IP address can make per minute (login: 20/min; everything else: 120/min). Stops brute-force guessing.
Environment variable	A secret/setting kept outside the code (database password, JWT secret). Stored in `.env`, which is never uploaded to GitHub.
HTTP status codes	200 OK · 201 created · 400 bad input · 401 not logged in/invalid token · 404 not found · 409 conflict (duplicate email) · 429 too many requests · 500 server error.
Serverless	Hosting where code runs only when a request arrives (Vercel). No server to babysit.
Expo / APK	Expo builds the React Native app; an APK is the installable Android file.
SecureStore	Phone's encrypted storage (backed by the Android Keystore) where the login pass is kept.
ESM	The modern JavaScript module system (`import ... from`). Our backend uses it natively, which is why import lines end with `.js`.

4. Folder-by-folder map

```
ProjectManagementSystem/
├─ README.md ................ project overview + setup (what a new developer reads first)
├─ DEMO.md .................. the 5-minute demo script
├─ FINAL_STATUS.md .......... test/build/deploy status report
├─ docker-compose.yml ....... one-command local stack (database + API) using Docker
├─ render.yaml .............. alternative deployment blueprint (Render)
├─ start-demo.bat ........... double-click helper: starts local API + website
├─ docs/
│   ├─ API.md ............... full API documentation (every endpoint)
│   └─ ER-DIAGRAM.md ........ database diagram
│
├─ backend/  (NestJS API)
│   ├─ prisma/
│   │   ├─ schema.prisma .... THE database design (User, Project, Task)
│   │   └─ migrations/ ...... saved SQL that builds the tables
│   ├─ src/
│   │   ├─ main.ts .......... program start: prefix /api, validation, CORS, logger
│   │   ├─ app.module.ts .... plugs all modules together + global rate limiter
│   │   ├─ app.controller.ts  health check: GET /api
│   │   ├─ common/
│   │   │   ├─ rate-limit.ts ................ rate limiter (guard + decorators)
│   │   │   └─ request-logger.middleware.ts . logs each request
│   │   ├─ prisma/ .......... database connection service
│   │   ├─ auth/ ............ register, login, logout, me, JWT checking
│   │   │   ├─ dto/ ......... shape rules for register/login bodies
│   │   │   ├─ guards/ ...... "must be logged in" checkpoint
│   │   │   └─ strategies/ .. how a JWT is verified
│   │   ├─ projects/ ........ project CRUD (+ dto/)
│   │   ├─ tasks/ ........... task CRUD (+ dto/)
│   │   └─ dashboard/ ....... the five statistics
│   ├─ scripts/
│   │   ├─ api-smoke-test.mjs  42 automatic checks against a running API
│   │   └─ seed-demo.mjs ..... creates demo data through the API
│   ├─ Dockerfile, jest.config.js, package.json, .env.example
│
├─ frontend/ (Next.js website)
│   └─ app/
│       ├─ page.tsx ......... the whole web app (login, dashboard, projects, tasks)
│       ├─ layout.tsx ....... page title, fonts
│       └─ globals.css ...... styling base (Tailwind)
│
└─ mobile/ (Expo app)
    ├─ app.json ............. app name, Android package id, plugins
    ├─ eas.json ............. APK build profile
    └─ src/
        ├─ app/ ............. the screens: login, register, index (dashboard), projects, tasks,
profile, _layout
        ├─ components/ ...... reusable pieces: Button, Input, Card, badges, BottomNavigation
        ├─ context/AuthContext.tsx . login state for the whole app
        ├─ services/ ........ api.ts (all network calls), auth.ts, authStorage.ts, projects.ts,
tasks.ts, dashboard.ts
        ├─ types/index.ts ... data shapes
        └─ constants/colors.ts
```

Files named `.env` hold secrets and are deliberately excluded from GitHub. `.env.example` lists the variable names without values.

5. The database (the filing cabinet)

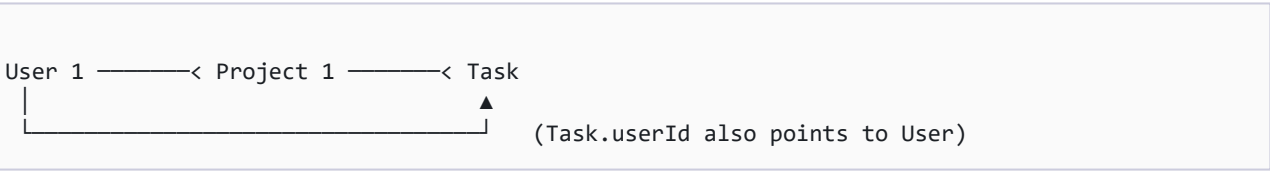
File: `backend/prisma/schema.prisma`. This one file describes the entire database. Prisma turned it into SQL (`backend/prisma/migrations/…/migration.sql`).

Three tables

Table	Columns	Notes
User	id (PK), fullName, email (UNIQUE), password (bcrypt hash), createdAt	Unique email enforced by the database itself (`User_email_key`).
Project	id, name, description?, status (enum), startDate?, endDate?, createdAt, **userId (FK→User)**	Status default NOT_STARTED. "?" means optional.
Task	id, name, description?, priority (enum), status (enum), dueDate?, createdAt, **projectId (FK→Project)**, **userId (FK→User)**	Priority default MEDIUM, status default PENDING.

Relationships (the "linked drawers")

- One **User** has many **Projects** (`Project.userId → User.id`).
- One **Project** has many **Tasks** (`Task.projectId → Project.id`).
- One **User** also has many **Tasks** directly (`Task.userId → User.id`). This makes "show me all my tasks" and "is this mine?" checks fast and simple.
- All three links are **ON DELETE CASCADE**: delete a project → its tasks vanish; delete a user → everything of theirs vanishes. No orphan rows.



The three enums

Enum	Allowed values
ProjectStatus	NOT_STARTED, IN_PROGRESS, COMPLETED (shown as "Not Started / In Progress / Completed")
TaskStatus	PENDING, IN_PROGRESS, COMPLETED
TaskPriority	LOW, MEDIUM, HIGH

Snippet from schema.prisma

```
model Task {
  id          Int          @id @default(autoincrement())
  name        String
  description String?
  priority    TaskPriority @default(MEDIUM)
  status      TaskStatus   @default(PENDING)
  dueDate     DateTime?
  createdAt   DateTime     @default(now())

  projectId   Int
  project     Project      @relation(fields: [projectId], references: [id], onDelete: Cascade)
  userId      Int
  user        User         @relation(fields: [userId], references: [id], onDelete: Cascade)
}
```

> **Say:** "The design is normalised: each fact is stored once. A task stores only the *id* of its project, not a copy of the project's name. Foreign keys are enforced by PostgreSQL, so the data can't become inconsistent."

Migrations: run `npx prisma migrate deploy` and Prisma executes the saved SQL to build all tables on any empty database. That's how the hosted database was set up.

6. The backend, file by file

NestJS organises code into **modules**. Each feature (auth, projects, tasks, dashboard) has: a **controller** (receives the request, decides which function runs), a **service** (does the real work), and **DTOs** (describe valid input). Think: controller = receptionist, service = specialist.

6.1 `src/main.ts` — the starting switch

Runs first. It does these things in order:

1. `import 'dotenv/config'` — loads the secrets from the `.env` file into memory.
2. Refuses to start if `JWT_SECRET` is missing (a safety net so we never run with an unsigned/empty secret).
3. `app.use(requestLogger)` — every request gets a log line.
4. `setGlobalPrefix('api')` — all addresses start with `/api`, as the assignment requires.
5. `useGlobalPipes(new ValidationPipe({ whitelist, forbidNonWhitelisted, transform }))` — the automatic input checker (explained in 6.7).
6. `enableCors(...)` — only the web addresses listed in `CORS_ORIGIN` may call the API from a browser.
7. Listens on `0.0.0.0` and `PORT` (default 5000). `0.0.0.0` means "accept connections from other devices", needed so a phone on Wi-Fi can reach the PC.

```
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,            // only fields declared in the DTO are allowed
    forbidNonWhitelisted: true, // extra/unknown fields make the request fail with 400
    transform: true,            // convert "5" to 5, trim/lowercase emails (see DTOs)
  }),
);
```

6.2 `src/app.module.ts` — plugging it all together

Lists every feature module (Prisma, Auth, Projects, Tasks, Dashboard), the health-check controller, and registers the **rate limit guard globally** (`APP_GUARD`) so it protects every route automatically.

6.3 `src/prisma/prisma.service.ts` — the database connection

A small class that extends `PrismaClient`. On startup it connects to PostgreSQL (`$connect()`), logs "Database connected", and on shutdown disconnects. Every service receives it ("injection") and uses it as `this.prisma.task.findMany(...)`. The database address comes from `DATABASE_URL` in the environment.

6.4 Authentication — `src/auth/`

File	Job
`auth.controller.ts`	Defines the 4 endpoints: `POST register`, `POST login`, `POST logout`, `GET me`. Register and login also carry a stricter rate limit (20/min). `me` carries the login guard.
`auth.service.ts`	The logic: register, login, getMe.

`dto/register.dto.ts`, `dto/login.dto.ts`	Input rules.
`auth.module.ts`	Configures JWT: secret from env, expiry from `JWT_EXPIRES_IN` (default 1 day).
`strategies/jwt.strategy.ts`	Teaches the system how to verify a token: read it from the `Authorization: Bearer` header, `ignoreExpiration: false` (expired = rejected), check the signature with the secret, and hand back `{ userId, email }`.
`guards/jwt-auth.guard.ts`	The reusable checkpoint `JwtAuthGuard`. Put it on a controller and every route behind it demands a valid token (otherwise automatic 401).

Register, step by step (`AuthService.register`)

1. Receive `fullName, email, password` (already validated; email already trimmed and lower-cased).
2. Look for an existing user with that email. If found → throw `ConflictException` → HTTP **409** "Email is already registered".
3. `bcrypt.hash(password, 10)` — scramble the password (the 10 is the "work factor": it makes each guess deliberately slow).
4. Save the user with the **hash**, never the plain password.
5. Create a JWT containing `sub` (user id) and `email`, valid for 1 day.
6. Reply with `{ message, token, user: { id, fullName, email } }` — note the password hash is **not** included.

```
const hashedPassword = await bcrypt.hash(password, 10);
const user = await this.prisma.user.create({
  data: { fullName, email, password: hashedPassword },
});
const token = this.jwtService.sign({ sub: user.id, email: user.email });
return { message: 'Registration successful', token,
         user: { id: user.id, fullName: user.fullName, email: user.email } };
```

Login (`AuthService.login`)

1. Find the user by email. Not found → 401 "Invalid email or password".
2. `bcrypt.compare(typedPassword, storedHash)`. Wrong → the **same** 401 message (so an attacker can't tell whether the email exists).
3. Correct → sign and return a new JWT.

Me (`GET /api/auth/me`)

Guard checks token → strategy puts `userId` on the request → `getMe` selects **only** id, fullName, email, createdAt (the `select` list makes it impossible to leak the password column).

Logout

JWTs are "stateless": the server keeps no list of active logins. Logout therefore means the **client throws its token away** (web removes it from localStorage; mobile deletes it from SecureStore). The endpoint exists (required by the assignment) and replies "Logout successful". The token also expires by itself after 1 day.

> **If asked "so is logout real?"** — "Yes for the user: the token is deleted on the device, so they are logged out. The token isn't revoked on the server because JWTs are stateless; adding revocation would need a blacklist table or refresh tokens, which is listed as a bonus feature we didn't build."

6.5 Projects — `src/projects/`

`projects.controller.ts` has `@UseGuards(JwtAuthGuard)` at the top, so **every** project route needs a valid token. Routes: `POST /projects`, `GET /projects` (with `?search=` and `?status=`), `GET /projects/:id`, `PUT /projects/:id`, `DELETE /projects/:id`. The controller pulls the user's id from the verified token (`request.user.userId`) and passes it to the service — **the user id never comes from the request body**, so a user cannot pretend to be someone else.

The ownership trick (the most important security idea in the code) in `projects.service.ts`:

```
async findOne(userId: number, id: number) {
  const project = await this.prisma.project.findFirst({
    where: { id, userId },          // the id AND the owner must both match
  });
  if (!project) throw new NotFoundException('Project not found');
  return project;
}
// update() and remove() call findOne() first, so they inherit the same check.
```

If you ask for project #7 and it belongs to somebody else, the query finds nothing and you get **404 "not found"** — the same answer as if it didn't exist, so nobody can even discover that other users' records exist.

Listing with search and filter:

```
where: {
  userId,                                          // only my projects
  ...(search ? { name: { contains: search, mode: 'insensitive' } } : {}),  // name search, ignores
capitals
  ...(status ? { status } : {}),                   // optional status filter
},
orderBy: { createdAt: 'desc' },                    // newest first
```

The `status` query value is checked by `ParseEnumPipe` in the controller: `?status=BANANA` gets a clean **400** instead of crashing.

6.6 Tasks — `src/tasks/`

Same pattern as projects, with extras:

- **Creating a task** first checks the target `projectId` belongs to *you* (`project.findFirst({ id: projectId, userId })`); otherwise 404 — so you can't drop tasks into someone else's project.
- **Updating a task** that moves it to another project does the same check for the new project.
- **List filters:** `search` (name), `status`, `priority`, and `projectId` (this last one powers "show tasks under a project"). All are combined with `userId`.
- **"Mark complete"** is simply `PUT /tasks/:id` with `{"status":"COMPLETED"}` — one rule, no special endpoint.

```
// tasks.service.ts (list)
where: {
  userId,
  ...(projectId !== undefined ? { projectId } : {}),
  ...(search   ? { name: { contains: search, mode: 'insensitive' } } : {}),
  ...(status   ? { status }   : {}),
  ...(priority ? { priority } : {}),
},
```

6.7 Validation — the DTO files

Each endpoint that accepts a body has a DTO class with "decorators" (the `@Something()` labels) saying what is allowed. Example `create-task.dto.ts`:

```
export class CreateTaskDto {
  @IsString() @IsNotEmpty()  name: string;             // required, not blank
  @IsString() @IsOptional()  description?: string;
  @IsEnum(TaskPriority) @IsOptional() priority?: TaskPriority;  // only LOW/MEDIUM/HIGH
  @IsEnum(TaskStatus)   @IsOptional() status?: TaskStatus;      // only
PENDING/IN_PROGRESS/COMPLETED
  @IsDateString() @IsOptional() dueDate?: string;      // must be a real date
  @IsInt() projectId: number;                          // required whole number
}
```

Bad input	Result
Missing name / empty string	400 with message list
Email "bad"	400 "email must be an email"
priority "URGENT"	400 (not in the enum)
dueDate "tomorrow-ish"	400 (not a valid date)
Extra field like `"role":"admin"`	400 (`forbidNonWhitelisted`)
Password shorter than 6 or longer than 72	400
Id in URL not a number (`/tasks/abc`)	400 (`ParseIntPipe`)

The register/login DTOs also **trim and lower-case the email** (`@Transform`), so `Bob@Mail.com` and `bob@mail.com` are the same account.

6.8 Dashboard — `src/dashboard/dashboard.service.ts`

Runs five **COUNT** queries in parallel (`Promise.all`), each filtered by the logged-in `userId`:

Number	Query
totalProjects	count projects of this user
totalTasks	count tasks of this user
completedTasks	...with status COMPLETED
pendingTasks	...with status PENDING
projectsInProgress	projects with status IN_PROGRESS

Nothing is hard-coded: change a task's status and the next dashboard load shows different numbers because they are counted from the database each time.

6.9 Rate limiting — `src/common/rate-limit.ts`

Written by hand (about 70 lines, no extra library). A **guard** applied to every route:

1. Work out the caller's IP address (honouring the `x-forwarded-for` header used behind Vercel).
2. Keep a counter per IP *per route* for a one-minute window.
3. If the counter exceeds the limit → HTTP **429 "Too many requests"**.

Limits: **120 per minute** for general routes; **20 per minute** for `register` and `login` (via the `@RateLimit(...)` label on those two). The health-check route is exempt (`@SkipRateLimit()`). Both numbers can be changed with environment variables. We saw it work: the 19th rapid wrong login returns 429.

> **Honest detail:** counters live in the server's memory, which is fine for this assessment. In a big production system with many servers you'd keep counters in Redis. (We originally used the popular `@nestjs/throttler` library, but it isn't compatible with the way Vercel runs NestJS 12, so we replaced it with this small own implementation.)

6.10 Request logging — `src/common/request-logger.middleware.ts`

Middleware that prints one line per request, e.g. `GET /api/tasks 200 7ms`. It deliberately logs the path *without* the query string so search words are not recorded.

6.11 Error handling

NestJS converts thrown exceptions into clean JSON automatically: `NotFoundException` → 404, `ConflictException` → 409, `UnauthorizedException` → 401, validation failures → 400, rate limit → 429. Unexpected crashes → 500 with a generic message (no stack trace to the user).

6.12 Health check — `src/app.controller.ts`

`GET /api` returns `{"status":"ok","service":"project-management-api"}`. Hosting platforms use it to check the server is alive.

6.13 Why do import lines end in `.js`?

The newest NestJS (v12) is published as modern "ES modules". To run it on Vercel the backend is also built as modern modules, and that style requires imports like `from './app.module.js'` (even though the source file is `.ts` — it becomes `.js` after building). It looks odd but is simply a rule of that module system; nothing special happens.

7. Follow one click all the way through

Scenario: on the Tasks screen you click **"Complete"** on the task "Design homepage" (id 12). Here is every step, with the file where it happens.

#	What happens	Where
1	The button's click runs `markTaskComplete(task)`.	`frontend/app/page.tsx`
2	The browser sends `PUT https://…/api/tasks/12` with header `Authorization: Bearer <token>` and body `{"status":"COMPLETED"}`.	same function, uses `fetch`
3	The browser first asks the API "may this website call you?" (CORS). The API answers yes because the site is in `CORS_ORIGIN`.	`main.ts`
4	The request logger starts a timer.	`request-logger.middleware.ts`
5	Rate-limit guard: has this IP made more than 120 requests this minute? No → continue.	`rate-limit.ts`
6	Route matched: `PUT /api/tasks/:id` → `TasksController.update`. `JwtAuthGuard` verifies the token's signature and expiry and loads `userId`.	`tasks.controller.ts`, `jwt.strategy.ts`
7	`ParseIntPipe` turns "12" into the number 12. `ValidationPipe` checks the body against `UpdateTaskDto` ("COMPLETED" is a valid status).	`update-task.dto.ts`
8	`TasksService.update(userId, 12, dto)` runs `findOne`: "is there a task 12 owned by this user?" If not → 404, stop.	`tasks.service.ts`
9	Prisma sends a safe `UPDATE "Task" SET status='COMPLETED' WHERE id=12` to PostgreSQL.	Prisma → database
10	The updated task goes back as JSON with status 200. The logger prints `PUT /api/tasks/12 200 9ms`.	—
11	The website calls `loadTasks()` and `loadDashboard()` again; the list shows "Completed" and the dashboard's Completed/Pending counts change.	`page.tsx`

The mobile app does exactly steps 2–10 too, from `mobile/src/app/tasks.tsx → handleComplete → updateTask() → authenticatedRequest()`. Because the database is shared, pulling down to refresh on the other device shows the change.

8. The website (frontend)

Folder `frontend/app/`. It is a Next.js app. The whole interface is in **one file,** `page.tsx` (about 1,900 lines) split into small components. It starts with `'use client'`, meaning it runs in the browser and keeps state with React's `useState`.

8.1 Key ideas

- **State** = the app's memory: `token`, `user`, `projects`, `tasks`, `dashboard`, current page, search text, filter values, modal open/closed, form values, loading flags, error text. When state changes, React redraws the screen.
- **The API address** comes from `NEXT_PUBLIC_API_URL` (locally `http://localhost:5000/api`; on Vercel the live API). Only a public URL is placed there — never a secret.
- **Token storage:** after login the token is saved in the browser's `localStorage` under the name `token` so a page refresh keeps you logged in. Logout deletes it.
- **Protected screens:** if there is no token, only the login/register screen renders. The real protection is on the backend (every data request needs a valid token); the front-end check is just for convenience.

8.2 The functions inside `Home()` (the main component)

Function	What it does
`useEffect` (first)	On page load: if a token exists in localStorage, restore the session by loading user data.
`useEffect` (filters)	Whenever a task filter dropdown (status/priority/project) or the project status filter changes, fetch again so results always match.
`submitAuth`	Handles both Sign in and Create account (mode switch). Posts to `/auth/login` or `/auth/register`, saves the token, clears the password field, loads data. Shows backend error messages (joins lists of messages).
`loadUserData`	Fetches `/auth/me`, `/dashboard`, `/projects`, `/tasks` together; if any returns 401, it logs out.
`loadDashboard / loadProjects / loadTasks`	Each calls its endpoint, adding `search`, `status`, `priority`, `projectId` as query parameters from the current filter state.
`openCreateProject / openEditProject / saveProject / deleteProject`	Open the modal blank or pre-filled; save posts (create) or puts (edit); delete asks "are you sure?" first (`window.confirm`) then calls DELETE.
`openCreateTask / openEditTask / saveTask / deleteTask`	Same for tasks, including project, priority, status, due date.
`markTaskComplete`	PUT status COMPLETED, then reload tasks and dashboard.
`logout`	Tells the API (informational), removes the token, clears all state → back to login screen.

8.3 The smaller components (bottom of the file)

Component	Purpose
`Dashboard`	The five stat cards (`StatCard`) plus a "Recent Projects" list.
`ProjectsPage`	Search box, status filter, "+ New Project", project cards with Edit/Delete.
`TasksPage`	Search, Project / Status / Priority dropdowns, "+ New Task", task cards with Edit / Complete / Delete.
`StatusBadge` / `PriorityBadge`	Coloured labels.
`EmptyState`	Friendly message when a list is empty.
`formatDate`	Turns a database timestamp into readable text.

8.4 Assignment requirements on the web

Requirement	How it's met
Responsive design	Tailwind CSS responsive classes (`md:` etc.): sidebar hides on small screens, grids collapse.
Form validation	Browser rules (`required`, `type="email"`, password `minLength`) plus the backend's full validation, whose messages are shown on screen.
Loading indicators	Buttons change to "Please wait… / Saving..." and are disabled while a request runs.
Error handling	Failed requests show a red message; expired token (401) logs the user out.
Component structure	Separate components per page and per small widget (see above).

9. The mobile app

Folder `mobile/`. Built with **Expo Router**: every file inside `src/app/` is a screen, and the file name is its address.

9.1 Screens

File	Screen
`_layout.tsx`	Root: wraps the app in `AuthProvider`. Shows a spinner while the stored login is being restored; then shows **only login/register** if not logged in, or **only the app screens** if logged in.
`login.tsx`, `register.tsx`	Forms with validation and error alerts.
`index.tsx`	Dashboard: five stat cards, recent tasks, active projects, pull-to-refresh.
`projects.tsx`	Project list, search, status chips, create/edit modal, delete with confirmation, pull-to-refresh.
`tasks.tsx`	Task list, search, filter chips for project / status / priority, create/edit modal (choose project, priority, status), Complete button, delete with confirmation, pull-to-refresh.
`profile.tsx`	Shows the account from `/auth/me`; logout button.

9.2 The service layer — `src/services/`

- **`api.ts`** — the single place that makes network calls.
  - `API_URL` comes from `EXPO_PUBLIC_API_URL`, defaulting to the live API.
  - `apiRequest` sends the request, parses the reply, and converts problems into a friendly `ApiError`. A network failure (no internet) becomes "Unable to connect to the server. Please check your internet/Wi-Fi connection" — so the app shows a message instead of crashing.
  - `authenticatedRequest` reads the token, refuses to call if there is none, and adds the `Authorization: Bearer` header.
  - **401 handling:** if a request that carried a token comes back 401, the token is deleted and "Your session has expired. Please log in again." is raised. (A wrong password at login is a 401 too, but without a token, so it correctly shows "Invalid email or password" instead.)
- **`authStorage.ts`** — saves/reads/deletes the token. On Android/iOS it uses **expo-secure-store** (encrypted, Android Keystore). Only in a desktop-browser preview, where SecureStore doesn't exist, it falls back to localStorage.
- `auth.ts, projects.ts, tasks.ts, dashboard.ts` — one small function per API call (`login`, `getTasks(filters)`, `createTask`, `updateTask` …).

9.3 Login state — `context/AuthContext.tsx`

One object shared by all screens: `user`, `token`, `loading`, and the actions `login`, `register`, `logout`, `restoreSession`.

1. **App opens** → `restoreSession`: read the token from SecureStore. None → show login. Present → call `/auth/me`.
2. **/auth/me succeeds** → we know the token is still valid → show the app. **Fails with 401** → delete the token, show an alert "Session expired", go to login.
3. Screens check `authLoading`, `user` and `token` before fetching, so **no protected request is sent before the login has been restored** (the assignment prompt specifically warns about this).

9.4 Mobile requirements

Requirement	How it's met
Same backend & database	Same `/api` endpoints; no separate server.
Secure token storage	expo-secure-store (Android Keystore).
Expired token → login + message	401 handling above.
No network → clear message	`ApiError` network branch + error cards/alerts.
Pull-to-refresh	`RefreshControl` on dashboard, projects, tasks, profile.
Loading / empty / error states	Components `LoadingState`, `EmptyState`, `ErrorState` in `components/ui.tsx`.
Create/edit/delete tasks, change status & priority, mark complete, search, filter	`tasks.tsx` (modal form + chips + Complete button).

9.5 Building the Android app (APK)

`app.json` holds the app name and Android package `com.projectmanagement.mobile`. `eas.json` defines the `preview` profile with `buildType: apk` and sets `EXPO_PUBLIC_API_URL` to the live API. Command: `npx eas-cli@latest build -p android --profile preview` (needs an Expo account login). For testing without an APK, run `npx expo start` and open it with the Expo Go app.

10. Security: what protects the system, and where

Threat / requirement	Defence	Where to point at in the code
Stolen database reveals passwords	Only bcrypt hashes are stored (cost 10)	`auth.service.ts` → `bcrypt.hash`
Password leaks in a response	Responses are built from chosen fields only; `/me` uses `select`	`auth.service.ts`
Anonymous access to data	JWT guard on projects, tasks, dashboard, me	`@UseGuards(JwtAuthGuard)`
Forged or old token	Signature check with secret; expiry enforced (1 day)	`jwt.strategy.ts`, `auth.module.ts`
Reading/changing other users' data	Every query filtered by `userId` from the token; foreign records → 404	`projects.service.ts`, `tasks.service.ts`
Adding a task to someone else's project	Project ownership check on create and on move	`tasks.service.ts`
SQL injection	Prisma parameterised queries; no raw SQL anywhere	all `this.prisma.…` calls
Bad / malicious input	DTO validation with whitelist; unknown fields rejected; enum + id checks	`dto/*.ts`, `main.ts`
Password guessing (brute force)	20 login/register attempts per minute per IP (then 429)	`rate-limit.ts`, `auth.controller.ts`
Other websites calling the API from a browser	CORS allow-list (our Vercel site + localhost)	`main.ts`
Secrets in the code / on GitHub	Environment variables; `.env` git-ignored; scan found none committed	`.env.example`, `.gitignore`
Server starts with no signing key	Startup refuses without `JWT_SECRET`	`main.ts`
Token on phone readable by other apps	Android Keystore via SecureStore	`authStorage.ts`
Error pages leaking internals	Nest returns generic messages for crashes	built into NestJS

> **Be upfront about:** web stores its token in `localStorage` (normal for a Bearer-token SPA; the assignment's secure-storage rule is specifically for mobile). A cookie-based approach with refresh tokens is a possible future upgrade.

11. Testing: how we know it works

11.1 The API smoke test — `backend/scripts/api-smoke-test.mjs` (42 checks)

Run with `npm run test:api` (server must be running). It creates two throwaway users and checks, over real HTTP:

- Registration works; duplicate email → 409; invalid body → 400; extra fields → 400.
- Login works (even with different capitalisation of the email); wrong password → 401.
- `/me` works with a token, not without (401), not with garbage (401); no password in any response.
- Project and task: create, read, update, delete, search, status/priority/project filters, invalid enum → 400, invalid id → 400, missing → 404.
- Mark complete works; dashboard numbers equal the real data.
- **Ownership:** user B trying to read/update/delete user A's project or task gets 404 every time; B's dashboard shows zeros.
- Deleting a project also removes its tasks (cascade).

Result: 42 of 42 passed — against the local server and against the live hosted server.

11.2 Unit tests — `backend/src/**/*.spec.ts` (10 tests)

Run with `npm test`. They test the services with a fake database: passwords are stored hashed and not returned; duplicate email rejected; wrong password and unknown user give the same error; task/project queries are always scoped to the user; foreign records give 404; dashboard counts are scoped by user.
Result: 10 of 10 passed.

11.3 Other checks that were run

- Expired token and wrong-signature token → 401 (tested with generated tokens).
- 25 rapid wrong logins → 429 after the limit.
- CORS: a request from our site gets permission; from a random site gets none.
- Builds: backend build, website production build, mobile type-check, mobile web export, Expo doctor (20/20 checks) — all passed.
- Browser test on the live website: log in with the demo account and see real dashboard numbers.

> **Not tested:** the mobile screens on a physical phone/emulator (this was verified by reading code and type-checking), and the Docker image build. Say so honestly if asked.

12. Deployment: how it got onto the internet

1. **Database:** a hosted PostgreSQL was created; `npx prisma migrate deploy` built the three tables there.
2. **Backend:** deployed to Vercel as a serverless function from the `backend/` folder. Environment variables set in Vercel: `DATABASE_URL`, `JWT_SECRET` (a random 96-character secret generated for production), `CORS_ORIGIN`. The build runs `prisma generate` (creates the database client code).
3. **Website:** deployed to Vercel from `frontend/` with `NEXT_PUBLIC_API_URL` pointing at the live API.
4. **Demo data:** `seed-demo.mjs` filled the live database *through the API* (not by touching the database), creating the demo user, 3 projects (one In Progress, one Not Started, one Completed) and 6 tasks with mixed priorities and statuses.
5. **Mobile:** configured to use the live API (`eas.json` and the default in `api.ts`).

Environment variables (what each one is)

Variable	Where	Meaning
DATABASE_URL	backend	Address + login of the database (secret)
JWT_SECRET	backend	Key used to sign tokens (secret)
JWT_EXPIRES_IN	backend	Token lifetime (default 1d)
PORT	backend	Local port (default 5000)
CORS_ORIGIN	backend	Allowed website addresses
RATE_LIMIT_PER_MINUTE / AUTH_RATE_LIMIT_PER_MINUTE	backend	120 / 20 by default
NEXT_PUBLIC_API_URL	frontend	Public API address (not secret)
EXPO_PUBLIC_API_URL	mobile	Public API address (not secret)

> **Database note:** the hosted database was created with a free "try it" service. It is temporary unless claimed. The claim link and a deadline are in the final report message; claiming (one click, free) makes it permanent. If it ever expired you would create a new PostgreSQL, set `DATABASE_URL` in Vercel, run `npx prisma migrate deploy`, and run `npm run seed:demo`.

Running everything locally

```
# 1. make sure PostgreSQL is running (Windows service postgresql-x64-18)
# 2. double-click start-demo.bat   (starts API on :5000 and website on :3000)
#    or by hand:
cd backend  && npm run build && npm run start:prod
cd frontend && npm run build && npm run start
# 3. open http://localhost:3000
```

13. Requirement checklist: assignment PDF vs. what we built

Assignment item	Status	Where / proof
Register / login / logout; full name, email, password	Done	`auth/`; web & mobile forms
Unique email	Done	DB unique index + 409 error
Passwords never stored in plain text	Done	bcrypt
Stay logged in until logout/expiry	Done	token saved on device; 1-day expiry
One account on web and mobile	Done	same API
Projects: create/view/edit/delete/list; fields name, description, status, start, end, created	Done	`projects/`
Tasks inside projects: create/edit/delete/complete/view under a project; all fields	Done	`tasks/`; project filter on web + mobile
Dashboard (5 numbers, per user)	Done	`dashboard/`
Search projects/tasks by name; filter projects by status; tasks by status & priority	Done	query params + UI controls
Mobile: all listed abilities, refresh, secure storage, expiry message, no-network message	Done (code-verified)	`mobile/src`
Web: React/Next, responsive, validation, loading, errors	Done	`frontend/app/page.tsx`
Backend: REST, routes, middleware, error handling, logging, clean structure, CORS	Done	NestJS modules; logger; CORS list
Database: relational, foreign keys, normalised	Done	`schema.prisma`
Security: bcrypt, protected APIs, authorization, validation, SQL injection, rate limiting on auth	Done	Section 10
All 15 required endpoints	Done	`docs/API.md`
Documentation: setup, env vars, DB setup, API docs, run mobile against deployed backend	Done	`README.md`, `docs/`
Submission: public GitHub	Done	github.com/AdityaParthasarathy/project-management-system

Submission: ER diagram, API docs, README	Done	`docs/ER-DIAGRAM.md`, `docs/API.md`, `README.md`
Submission: deployment URLs (web + backend)	Done	Vercel URLs on the cover page
Submission: Android APK / Expo link	Pending (one login)	Config is ready; run the EAS build command after logging in to Expo (section 9.5). A local Windows build is blocked by Windows' 260-character path limit.
Submission: 5-minute screen recording	You	Script in section 14
Bonus: Docker	Done (not build-tested)	`backend/Dockerfile`, `docker-compose.yml`
Bonus: unit/integration tests	Done	10 unit tests + 42-check API test
Bonus: CI pipeline	File provided	`docs/github-actions-ci.yml` (copy into `.github/workflows/`; GitHub token lacked permission to push it)
Bonus not done: pagination, sorting, audit logs, roles, refresh tokens, push notifications, offline mode, shared types	Not done	Say honestly: optional, prioritised the core

14. The 5-minute demo, with what to say

Time	Do	Say
0:00	Show the website login page.	"This is ProjectFlow: Next.js website and an Expo Android app, one NestJS API, one PostgreSQL database."
0:30	Register a new account (or log in as demo).	"Password is hashed with bcrypt; I get back a JWT that expires in a day."
1:00	Dashboard.	"These five numbers are counted from the database for my user only."
1:30	Projects → New Project: name, description, status In Progress, dates.	"Create. The backend validated the input and saved it with my user id."
2:00	Tasks → New Task in that project, priority High.	"A task belongs to a project and to me."
2:30	Edit the task: priority Medium, status In Progress.	"Update goes through `PUT /api/tasks/:id`."
3:00	Click Complete.	"Marking complete is just a status change."
3:30	Back to Dashboard.	"Completed and pending counts changed — nothing is hard-coded."
4:00	Search a task; use Project, Status, Priority filters; filter projects by status.	"Filters become query parameters; the database does the filtering."
4:30	Phone: log in with the *same* account, pull to refresh. Create a task on the phone, refresh the web page.	"Same backend, same database, so both always agree. The phone keeps the token in the Android Keystore."
5:00	(Optional) Log in as a second user: empty dashboard.	"Every query is filtered by user id; other users' data is invisible — even by guessing ids (404)."

> **Before recording:** open the website a minute early and log in once. The free hosted backend "wakes up" on its first request and that first login can take 5–10 seconds; after that it is fast. If you demo locally (localhost:3000) there is no such delay.

15. Questions the reviewer might ask — with simple answers

About the system overall

1. Explain the architecture in one minute.
Two clients (website, Android app) call one REST API built with NestJS. The API checks identity, validates input, applies rules, and uses Prisma to read/write a PostgreSQL database with three tables: User, Project, Task.

2. Why one backend for both apps?
The assignment requires it, and it avoids duplicate logic: the rules (ownership, validation) live in one place and both apps get identical behaviour and data.

3. Why NestJS rather than plain Express?
It gives a clear structure (modules/controllers/services), built-in validation, guards and dependency injection. NestJS is on top of Express internally, so it satisfies "Node with Express or NestJS".

4. Why PostgreSQL / why relational?
The data has clear relationships (user → projects → tasks). A relational database enforces them with foreign keys and gives consistent counts for the dashboard.

5. What is Prisma and why use it?
An ORM: code-friendly queries, type-safety, migrations, and automatic parameterisation which prevents SQL injection.

Authentication & security

6. How does login work?
Client posts email+password. The server finds the user, compares with `bcrypt.compare`, and if it matches returns a JWT. The client sends it as `Authorization: Bearer …` on later requests.

7. What is in the JWT?
The user id (`sub`), email, issued-at and expiry time, plus a signature made with the server's secret. It is signed, not encrypted: don't put secrets in it (we don't).

8. What happens when the token expires?
The server rejects it with 401. The mobile app deletes the token, shows "session expired", and goes to the login screen. The website logs out.

9. How are passwords protected?
bcrypt with salt (built into bcrypt) and cost factor 10. Only the hash is stored. Responses never include it.

10. How do you stop one user seeing another's data?
The user id is taken from the verified token (never from the request) and every database query includes it in the `where`. Someone else's record simply isn't found → 404.

11. Why 404 instead of 403 for someone else's record?
403 would confirm the record exists. 404 reveals nothing.

12. How do you prevent SQL injection?
We never build SQL strings. Prisma sends user input as bound parameters, so input can't become a command.

13. What is rate limiting here?
A guard counting requests per IP per minute: 20 for register/login, 120 for everything else; beyond that, HTTP 429.

14. What is CORS and how is it set?
A browser rule that blocks a website from calling an API unless the API allows it. `CORS_ORIGIN` lists our Vercel site and localhost; other sites get no permission.

15. Where are the secrets?
In environment variables (`.env` locally, Vercel settings in production). `.env` is excluded from git; `.env.example` shows the names only.

16. Is logout truly secure?
The token is deleted from the device; it cannot be revoked server-side (stateless JWT) but expires within a day. Refresh tokens/blacklist would be the upgrade.

17. Why is the web token in localStorage?
Simple and standard for a Bearer-token app. It can be read by scripts if the site had an XSS bug; React escapes content by default. Mobile uses the hardware-backed Keystore as the assignment requires.

18. What does `whitelist` / `forbidNonWhitelisted` do?
Only fields declared in the DTO are accepted; any extra field (e.g. someone sending `role: admin`) causes a 400. It blocks "mass assignment" attacks.

19. How do you validate inputs?
DTO classes with `class-validator` decorators; the global `ValidationPipe` runs them before our code executes.

20. Could someone guess ids like /projects/1,2,3?
They'd get 404 for everything not theirs, because ownership is part of the query.

Backend code

21. What is a controller vs a service?
Controller: maps a URL to a function and reads the request. Service: contains the business logic and database calls.

22. What is a guard?
A checkpoint run before a route. `JwtAuthGuard` allows only requests with a valid token; the rate-limit guard allows only requests under the limit.

23. What is a pipe?
A transformer/validator for incoming values: `ParseIntPipe` (id → number), `ParseEnumPipe` (checks a filter value), `ValidationPipe` (checks bodies).

24. What is middleware here?
The request logger that runs on every request and logs method, path, status and time. (Guards and pipes are also forms of middleware-like layers.)

25. What is dependency injection?
Classes declare what they need (e.g. `PrismaService`) in their constructor and NestJS provides it. It keeps code decoupled and makes testing easy (we inject a fake database in unit tests).

26. How is search implemented?
Prisma `contains` with `mode: 'insensitive'` on the name column → SQL `ILIKE '%text%'`, safely parameterised.

27. How is the dashboard computed?
Five `count` queries run together, each filtered by the user.

28. What happens if I delete a project that has tasks?
The foreign key is `ON DELETE CASCADE`; the database deletes the tasks too.

29. How does "mark complete" work?
It's an update that sets `status = COMPLETED`. No separate endpoint needed.

30. What is a migration?
A versioned SQL script describing a schema change. `prisma migrate deploy` applies pending ones, so any new database can be built identically.

31. How is error handling done?
Services throw typed exceptions (NotFound, Conflict, Unauthorized); Nest turns them into the right status code + JSON message. Validation errors list each problem.

32. What logging exists?
A request logger (method, path, status, duration), Nest's built-in logger for startup and database connection.

33. Why did you write your own rate limiter?
The standard library didn't work with NestJS 12's module format on Vercel's serverless runtime. The own version is ~70 lines, tested, and configurable.

34. Limitation of that limiter?
Counters are per server instance in memory; with many instances you'd use Redis.

Web / mobile

35. How does the website know you're logged in after refresh?
On load it reads the token from localStorage and fetches your data; if the API says 401 it logs out.

36. How does the mobile app keep the user logged in?
Token in SecureStore; on every launch `restoreSession` reads it and calls `/auth/me`.

37. How do you avoid calling the API before login is restored?
Screens wait for `authLoading` to finish and require `user` and `token` before fetching; the layout shows a spinner during restore.

38. What if there's no internet?
The fetch throws a network error; `api.ts` converts it to a readable message and screens show an error card/alert — never a blank screen.

39. How do changes sync between web and mobile?
There's no push sync; both read the same database. After a change, refreshing (pull-to-refresh on mobile, reload/reopen on web) shows it.

40. Why Expo?
Fast development, easy APK builds (EAS), SecureStore built-in. React Native lets us write one TypeScript codebase.

41. What's a React "state" and "effect"?
State is data that, when changed, redraws the screen. An effect is code that runs after the screen draws or when certain values change (e.g. re-fetch when a filter changes).

42. How do the filters work end-to-end?
Selecting a filter sets state → effect triggers a fetch like `/tasks?status=PENDING&priority=HIGH&projectId=3` → backend builds the `where` → returns only matches.

Process, testing, deployment, honesty

43. How did you test?
A 42-check automated API test (including ownership and validation), 10 unit tests with a fake database, plus build/type checks and a browser test of the live site.

44. Where is it deployed and how?
Both the website and API are on Vercel; the database is hosted PostgreSQL. Env vars hold the secrets; migrations are applied with `prisma migrate deploy`.

45. How would someone else run it?
README: install each folder, copy `.env.example` to `.env`, set `DATABASE_URL` and `JWT_SECRET`, run `npx prisma migrate deploy`, then start backend, web, and mobile. Or `docker compose up --build` for backend + database.

46. What would you improve with more time?
Refresh tokens and server-side logout, pagination and sorting, Redis-backed rate limiting, audit logs, split the big web page into separate files/routes, push notifications for due tasks, offline cache on mobile, end-to-end tests of the UI.

47. What was the hardest problem?
Getting the NestJS 12 (ES-module-only) backend to run on Vercel: the standard throttler library and CommonJS output failed there, so the backend was converted to native ES modules and the rate limiter was rewritten.

48. Did you use AI tools?
The assignment explicitly allows AI-assisted tools. Be ready to explain any file using this guide — that is the point of it.

49. Is it production-ready?
Not claimed: the assignment says it need not be, but it demonstrates good practice (hashing, validation, ownership, rate limiting, env secrets, migrations, tests, docs).

50. How do you know the data isn't fake?
Open the database or API: the dashboard is five COUNT queries; the demo data was created through the API by `seed-demo.mjs`.

16. Honest limitations (and good ways to say them)

Limitation	Suggested wording
Logout doesn't revoke the token server-side	"JWTs are stateless; the client discards the token and it expires in 1 day. Refresh tokens/blacklist are the next step."
Web UI is one large file	"It's organised into components inside one file for speed of delivery; the next step is splitting into routes and files."
Rate limiter is in-memory	"Perfect for one instance; Redis for many."
Mobile not tested on a physical phone by me during build	"Verified by type-checking, a web export and code review; I'd confirm on a device when installing the APK."
Hosted database is a free temporary one unless claimed	"Can be switched to any PostgreSQL by changing `DATABASE_URL`; the migrations recreate the tables."
First request after idle is slow (serverless cold start)	"Free-tier behaviour; subsequent requests are fast."
No pagination/sorting UI	"Optional bonus; lists are filtered and newest-first."
CI file is not active in the repository	"Provided in `docs/github-actions-ci.yml`; GitHub requires a special permission to push workflow files."

17. Cheat sheet

Item	Value
Live website	https://projectflow-web-tau.vercel.app
Live API health	https://projectflow-api-virid.vercel.app/api
GitHub	https://github.com/AdityaParthasarathy/project-management-system
Demo login (live and local)	demo@projectflow.test / DemoPass#2026
Local website / API	http://localhost:3000 / http://localhost:5000/api
Start local	double-click `start-demo.bat` (PostgreSQL must be running)
Run API test	`cd backend && npm run test:api`
Run unit tests	`cd backend && npm test`
Create demo data (via API)	`cd backend && npm run seed:demo`
Mobile dev	`cd mobile && npx expo start`
Build APK	`cd mobile && npx eas-cli@latest build -p android --profile preview`

Most important backend files to know	`prisma/schema.prisma`, `src/main.ts`, `src/auth/auth.service.ts`, `src/projects/projects.service.ts`, `src/tasks/tasks.service.ts`, `src/common/rate-limit.ts`
Most important web / mobile files	`frontend/app/page.tsx`; `mobile/src/services/api.ts`, `context/AuthContext.tsx`, `app/tasks.tsx`

Tip: in the code, search for the words "userId" to see ownership checks, "bcrypt" for password handling, "JwtAuthGuard" to see what is protected, and "RateLimit" for throttling.